

PUC MINAS

Bacharelado em Engenharia de Software



Relatório Técnico: Refatoração de Testes e Detecção de Test Smells

Disciplina: Testes de Software

Aluno: João Pedro Aguiar

Matrícula: 756491

1. Introdução e Contexto

No desenvolvimento moderno de software, a existência de uma suíte de testes automatizados é considerada indispensável para garantir a estabilidade e a corretude do sistema durante o ciclo de evolução do código. Contudo, a mera presença de testes ou a obtenção de métricas quantitativas elevadas, tais como 100% de cobertura de código, não são garantias absolutas de eficácia. Conforme evidenciado por conceitos de Testes de Mutação, testes estruturalmente fracos ou mal desenhados podem permitir a sobrevivência de "mutantes", falhando em sua missão principal de identificar regressões e defeitos na lógica de negócio.

A ineficácia de muitas suítes de testes está diretamente ligada aos chamados *Test Smells* (maus cheiros em testes). O termo descreve sintomas ou padrões de design inadequados no código de teste que sinalizam problemas arquiteturais ou de implementação mais profundos. Testes afetados por esses problemas tornam-se frágeis, obscuros, lentos para executar e complexos de manter, aumentando expressivamente a taxa de falsos positivos ou falsos negativos.

Este relatório técnico documenta as atividades de identificação manual e automatizada de *Test Smells* em uma API de gerenciamento de usuários. Para a detecção automática, utilizou-se a ferramenta de análise estática ESLint integrada ao plugin especializado para o framework Jest. Posteriormente, foi realizada a refatoração completa da suíte de testes aplicando-se rigorosamente o padrão de design **Arrange, Act, Assert (AAA)**, convertendo o código problemático em uma suíte limpa, legível, desacoplada e sustentável.

2. Análise de Test Smells Identificados

A suíte de testes original contida no arquivo `userService.smelly.test.js` foi submetida a uma inspeção detalhada. Foram catalogados três tipos principais de *Test Smells* que comprometiam severamente a qualidade e a confiabilidade do projeto:

2.1. Lógica Condicional de Teste (Conditional Test Logic)

Descrição e Contexto: Este *smell* ocorre quando o fluxo de execução de um teste deixa de ser linear e previsível, incorporando estruturas de controle típicas de código de produção, tais como loops (`for`, `while`), desvios condicionais (`if/else`) ou blocos de captura de exceções (`try/catch`). Na suíte analisada, este padrão manifestou-se em dois cenários:

- No teste de desativação de usuários, onde um loop `for...of` iterava sobre uma lista mista de usuários (comuns e administradores), e um bloco `if (!user.isAdmin)` alternava as asserções conforme o tipo de perfil.
- No teste de validação de menor de idade, onde utilizou-se uma estrutura `try/catch` para capturar o erro lançado pela regra de negócio.

Risco Associado: A introdução de lógicas complexas faz com que o próprio teste passe a necessitar de testes para garantir sua validade. O risco mais severo é a ocorrência de **falsos positivos**. No caso do bloco `try/catch`, se a validação de menor de idade for removida acidentalmente da lógica de negócio e o método parar de lançar a exceção, o código do teste nunca entrará no bloco `catch`.

Consequentemente, nenhuma asserção (expect) será disparada e a ferramenta considerará o cenário como bem-sucedido, camuflando um bug crítico.

2.2. Teste Ansioso (Eager Test)

Descrição e Contexto: O *Eager Test* manifesta-se quando um único método de teste tenta validar múltiplos comportamentos ou fluxos independentes de uma só vez. Na suíte original, o teste intitulado 'deve criar e buscar um usuário corretamente' acumulava duas ações de negócio complexas: primeiro, realizava a criação do usuário e verificava se o ID foi gerado; em seguida, no mesmo escopo e sem isolamento, usava esse ID para invocar o método de busca e validar os atributos do objeto retornado.

Risco Associado: Este padrão viola frontalmente o princípio de responsabilidade única do teste unitário, que determina que cada teste deve possuir apenas um motivo para falhar. Quando um teste ansioso falha, o diagnóstico imediato da causa raiz é severamente prejudicado, pois o desenvolvedor precisa investigar se o problema está na persistência inicial ou no mecanismo de consulta. Adicionalmente, caso ocorra um erro na primeira etapa (criação), a execução é interrompida imediatamente, impedindo a execução e validação da segunda etapa (busca), ocultando potenciais erros em cascata.

2.3. Teste Ignorado (Ignored / Disabled Test)

Descrição e Contexto: Caracteriza-se pelo uso de anotações ou modificadores fornecidos pelo framework de testes para desativar temporariamente a execução de um cenário, tais como `test.skip`, `xit` ou `xdescribe`. No arquivo analisado, o teste 'deve retornar uma lista vazia quando não há usuários' encontrava-se desabilitado sob a justificativa de uma pendência de implementação de funcionalidade (marcação "TODO").

Risco Associado: Testes ignorados representam uma dívida técnica silenciosa que tende a "apodrecer" (*test rotting*) à medida que a aplicação evolui. Ao desativar o teste, ele deixa de ser integrado aos ciclos de integração contínua (CI/CD), mascarando falhas estruturais. Quando o modificador `.skip` é removido futuramente, o teste quase invariavelmente falha devido a descompassos com as alterações acumuladas no código de produção, gerando retrabalho e falsa sensação de segurança sobre a real cobertura da suíte.

3. Processo de Refatoração

A estratégia adotada para a higienização da suíte de testes baseou-se na reescrita completa dos cenários no arquivo novo `userService.clean.test.js`, aplicando o padrão arquitetural **Arrange, Act, Assert (AAA)** e desmembrando lógicas complexas. O teste de desativação de perfis foi selecionado para ilustrar essa transformação:

Versão Original (Com Smells)	Versão Refatorada (Limpa - Padrão AAA)
<pre>test('deve desativar usuários se eles não forem administradores', () => { const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30); const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true); const todosOsUsuarios = [usuarioComum, usuarioAdmin]; for (const user of todosOsUsuarios) { const resultado = userService.deactivateUser(user.id); if (!user.isAdmin) { expect(resultado).toBe(true); const usuarioAtualizado = userService.getUserById(user.id); expect(usuarioAtualizado.status).toBe('inativo'); } else { expect(resultado).toBe(false); } } });</pre>	<pre>test('deve desativar um usuário comum com sucesso', () => { // Arrange const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30); // Act const resultado = userService.deactivateUser(usuarioComum.id); // Assert expect(resultado).toBe(true); const usuarioAtualizado = userService.getUserById(usuarioComum.id); expect(usuarioAtualizado.status).toBe('inativo'); }); test('não deve desativar um usuário administrador', () => { // Arrange const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true); // Act const resultado = userService.deactivateUser(usuarioAdmin.id); // Assert expect(resultado).toBe(false); });</pre>

Justificativa das Decisões de Refatoração

A reestruturação promoveu melhorias profundas na manutenibilidade do projeto por meio das seguintes ações:

1. **Separação de Casos de Uso:** O teste original misturava duas regras de negócio independentes. Ao criar dois testes específicos ('deve desativar um usuário comum com sucesso' e 'não deve desativar um usuário administrador'), isolou-se o comportamento de cada cenário, garantindo independência na execução.
2. **Eliminação de Lógica Condicional:** A remoção completa do laço for e do bloco condicional if/ else erradicou o risco de caminhos ocultos de execução e falsos positivos, resultando em um fluxo perfeitamente determinístico.
3. **Adoção Estrita do Padrão AAA:** Cada método agora possui blocos delimitados e comentados para preparação de dados (Arrange), execução da unidade sob teste (Act) e asserção dos resultados esperados (Assert), maximizando a legibilidade para qualquer membro da equipe técnica.

4. Relatório da Ferramenta de Análise Estática (ESLint)

A automação na detecção de falhas de qualidade foi estabelecida utilizando o ecossistema do ESLint v8 combinado ao plugin especializado eslint-plugin-jest. O arquivo de configuração .eslintrc.json determinou regras severas para coibir práticas prejudiciais, definindo erros de compilação para lógicas condicionais e alertas para testes ignorados.

Na primeira execução do comando `npx eslint .` sobre a suíte poluída, a ferramenta identificou de forma imediata 6 problemas críticos, divididos em 4 erros fatais e 2 avisos (warnings). O terminal apontou com precisão cirúrgica a linha e a coluna de cada ocorrência:

```
PS C:\Users\Joao\Documents\Faculdade\Testes de software\test-smelly> npx eslint .

C:\Users\Joao\Documents\Faculdade\Testes de
software\test-smelly\test\userService.smelly.test.js
 44:9  error  Avoid calling `expect` conditionally  jest/no-conditional-expect
 46:9  error  Avoid calling `expect` conditionally  jest/no-conditional-expect
 49:9  error  Avoid calling `expect` conditionally  jest/no-conditional-expect
 73:7  error  Avoid calling `expect` conditionally  jest/no-conditional-expect
 77:3  warning Tests should not be skipped  jest/no-disabled-tests
 77:3  warning Test has no assertions      jest/expect-expect

✖ 6 problems (4 errors, 2 warnings)
```

A introdução da análise estática altera radicalmente o paradigma de revisão de código, pois elimina a subjetividade humana da detecção de falhas elementares. Em vez de depender de revisões manuais demoradas em Pull Requests, o linter atua como um portão de qualidade automatizado e instantâneo na máquina do desenvolvedor, garantindo a aderência a padrões arquiteturais antes mesmo da execução da suíte de testes.

5. Conclusão

A execução prática deste projeto consolidou o entendimento de que métricas quantitativas isoladas, como a cobertura de código tradicional, são insuficientes para atestar a verdadeira confiabilidade de um software. Um sistema pode ostentar uma cobertura integral de suas linhas de comando enquanto sua suíte de testes permanece frágil, opaca e incapaz de flagrar regressões devido à incidência latente de *Test Smells*.

A eliminação sistemática de códigos ansiosos, testes ignorados e lógicas condicionais redundou em uma suíte higienizada, cujos benefícios imediatos incluem um tempo de diagnóstico drasticamente reduzido e o expurgo completo de comportamentos de falso positivo. A arquitetura linear promovida pelo padrão Arrange, Act, Assert simplifica a leitura do código, facilitando a integração de novos engenheiros e agilizando as janelas de manutenção.

Por fim, conclui-se que a combinação de testes unitários limpos com ferramentas de análise estática automatizada constitui um pilar inalienável para a sustentabilidade de projetos de software. Ao delegar ao linter a fiscalização contínua de padrões de design, a equipe de engenharia minimiza o acúmulo de dívidas técnicas e estabelece uma esteira de entrega contínua altamente resiliente, econômica e preparada para suportar o crescimento escalonável do produto.